

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..5

CUSTOMIZING THE UI

 Copy as Markdown

Customize the email sign-in UI

When using the email identity provider, you can customize the UI to your liking. You can use the `EmailSignInWidget` to display the email authentication flow in your own custom UI, or you can use the `EmailAuthController` to build a completely custom authentication interface.

INFO

The `SignInWidget` uses the `EmailSignInWidget` internally to display the email authentication flow. You can also supply a custom `EmailSignInWidget` to the `SignInWidget` to override the default behavior.

```
SignInWidget(  
  client: client,  
  emailSignInWidget: EmailSignInWidget(  
    client: client,  
    // Open on the login screen instead of the default registration screen  
    startScreen: EmailFlowScreen.login,  
    // A custom widget replaces the built-in handling, so pass your own callbacks.  
    onAuthenticated: () { /* ... */ },  
    onError: (error) { /* ... */ },  
  ),  
)
```

Using the `EmailSignInWidget`

The `EmailSignInWidget` handles the complete email authentication flow including login, registration, email verification, and password reset.

You can customize the widget's behavior using its constructor parameters:

```
EmailSignInWidget(  
  client: client,
```



```

client: client,
startScreen: EmailFlowScreen.login, // or startRegistration
// Customize the verification code configuration
// The default matches the default server verification code generation
verificationCodeConfig: VerificationCodeConfig(
  length: 6,
  keyboardType: TextInputType.number,
  allowedCharactersPattern: RegExp(r'[0-9]'),
  // The wait before the user can request a new code.
  resendCountdownDuration: Duration(minutes: 1),
),
// Custom email validation function. Throw InvalidEmailException so the
// widget shows the message. Other exceptions block sign-in silently.
emailValidation: (email) {
  if (!email.contains('@example.com')) {
    throw const InvalidEmailException('Only @example.com emails allowed');
  }
},
// Customize the password requirements
// If omitted, uses the default configuration below
passwordRequirements: [
  PasswordRequirement.minLength(12),
  PasswordRequirement.containsUppercase(),
  PasswordRequirement.containsLowercase(),
  PasswordRequirement.containsNumber(),
  PasswordRequirement.containsSpecialCharacter(),
],
onTermsAndConditionsPressed: () {
  // Open terms and conditions
},
onPrivacyPolicyPressed: () {
  // Open privacy policy
},
onAuthenticated: () {
  // Do something when the user is authenticated.
  //
  // NOTE: You should not navigate to the home screen here, otherwise
  // the user will have to sign in again every time they open the app.
},
onError: (error) {
  // Handle errors
},
)

```

Optionally, you can provide an externally managed `EmailAuthController` instance to the widget. A controller and a `client` are mutually exclusive, and `onAuthenticated` and `onError` belong on the controller in that case. Passing either alongside a controller trips an assertion, so a debug build throws. The controller carries the sign-in options, while `verificationCodeConfig`, `onTermsAndConditionsPressed`, and `onPrivacyPolicyPressed` stay on the widget.

```

EmailSignInWidget(
  controller: controller,
)

```

INFO

The terms and conditions and privacy policy checkbox on the registration screen are optional and disabled by default. The checkbox is shown as soon as you provide either `onTermsAndConditionsPressed` or `onPrivacyPolicyPressed`.

Customizing the default widget's appearance

Since the `EmailSignInWidget` uses the Material Design system, it will react to your app's Material theme. You can also wrap it in a `Theme` widget to apply a custom theme.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

Theme(
  data: Theme.of(context).copyWith(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
    // Use the AuthIdpTheme to customize the verification code input look
    extensions: <ThemeExtension<dynamic>>[
      AuthIdpTheme.defaultTheme(
        errorPinTheme: PinTheme(...),
      ),
    ],
  ),
  child: EmailSignInWidget(client: client),
)
```

Building a custom UI with the `EmailAuthController`

For more control over the UI, you can use the `EmailAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

final controller = EmailAuthController(
  client: client,
  startScreen: EmailFlowScreen.login,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
);
```

EmailAuthController state management

Your widget should render the appropriate screen based on the `currentScreen` property of the controller. You can also use the below state properties to build your UI:

```
// Check current screen
final currentScreen = controller.currentScreen;

// Check if loading
final isLoading = controller.isLoading;

// Check if authenticated
final isAuthenticated = controller.isAuthenticated;

// Get error message
final errorMessage = controller.errorMessage;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

The email authentication flow

The email authentication flow uses the following screens:

- `EmailFlowScreen.login` - Login with email and password
- `EmailFlowScreen.startRegistration` - Start registration with email
- `EmailFlowScreen.verifyRegistration` - Verify registration code
- `EmailFlowScreen.completeRegistration` - Set password to complete registration
- `EmailFlowScreen.requestPasswordReset` - Request password reset
- `EmailFlowScreen.verifyPasswordReset` - Verify password reset code
- `EmailFlowScreen.completePasswordReset` - Set new password

Navigating between screens

The controller handles navigation between screens automatically when calling the methods associated with each screen, but you can also navigate manually:

```
// Navigate to a specific screen
controller.navigateTo>EmailFlowScreen.startRegistration);

// Navigate back
if (controller.canNavigateBack) {
  controller.navigateBack();
}
```

```
}
```

EmailAuthController methods

The controller provides methods for each step of the authentication flow:

ⓘ INFO

The `EmailAuthController` already exposes internal text controllers for the email, password and verification code inputs. You can use these to build your own custom UI. All controllers are shared between login, registration and password reset flows for good UX. The `EmailAuthController` will handle the cleanup of the text controllers upon navigation as needed.

Login

```
// Set email and password in the controller's text controllers
controller.emailController.text = 'user@example.com';
controller.passwordController.text = 'password123';

// Call login
await controller.login();
```

Registration flow

The registration flow consists of three steps:

1. **Start Registration** - Request a verification code:

```
controller.emailController.text = 'user@example.com';
await controller.startRegistration();
// Controller navigates to verification screen
```

2. **Verify Registration Code** - Enter the verification code:

```
controller.verificationCodeController.text = '12345678';
await controller.verifyRegistrationCode();
// Controller navigates to password setup screen
```

3. **Complete Registration** - Set password:

```
controller.passwordController.text = 'securePassword123';
await controller.finishRegistration();
// User is now authenticated
```

Password reset flow

The password reset flow also consists of three steps:

1. Request Password Reset:

```
controller.emailController.text = 'user@example.com';  
await controller.startPasswordReset();  
// Controller navigates to verification screen
```

2. Verify Reset Code:

```
controller.verificationCodeController.text = '12345678';  
await controller.verifyPasswordResetCode();  
// Controller navigates to new password screen
```

3. Complete Password Reset:

```
controller.passwordController.text = 'newSecurePassword123';  
await controller.finishPasswordReset();  
// User is authenticated with new password
```

Resending verification codes

To resend a verification code:

```
await controller.resendVerificationCode();
```

This will resend the code based on the current screen (registration or password reset).